Addressing mode	Use of the operand
Immediate	The operand is the value to be used in the instruction; `SUB #48` is an example.
Direct	The operand is the address which holds the value to be used in the instruction; `ADD TOTAL` is an example.
Indirect	The operand is an address that holds the address which has the value to be used in the instruction.
Indexed	The operand is an address to which must be added the value currently in the index register (IX) to get the address which holds the value to be used in the instruction.

Table 6.06 Addressing modes

For immediate addressing there are three options for defining the value:

- #48 specifies the denary value 48
- #B00110000 specifies the binary equivalent
- #&30 specifies the hexadecimal equivalent

6.06 Assembly language instructions

We continue to consider a simple processor with a limited instruction set. The examples described here do not correspond directly to those found in the assembly language for any specific processor. Individual instructions will have a match in more than one real-life set. The important point is that these examples are representative. In particular, there are examples of the most common categories of instruction.

Data movement

These types of instruction can involve loading data into a register or storing data in memory. Table 6.07 contains a few examples of the format of the instructions with explanations.

Instruction opcode	Instruction operand	Explanation
`LDM`	`#n`	Immediate addressing. Load the number n to ACC.
`LDR`	`#n`	Immediate addressing. Load the number n to IX.
`LDD`	`<address>`	Direct addressing. Load the contents at the given address to ACC.
`LDI`	`<address>`	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
`LDX`	`<address>`	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
`MOV`	`<register>`	Move the contents of the accumulator to the given register (IX).
`STO`	`<address>`	Store the contents of ACC at the given address.

Table 6.07 Some instruction formats for data movement

The important point to note is that the mnemonic defines the instruction type including which register is involved and, where appropriate, the addressing mode. It is important to read the mnemonic carefully! The instruction will have an actual address where `<address>` is shown, a register abbreviation where `<register>` is shown and a denary value for `n` where `#n` is shown. The explanations use ACC to indicate the accumulator. For explanations of `LDD`, `LDI` and `LDX`, refer back to Table 6.07.

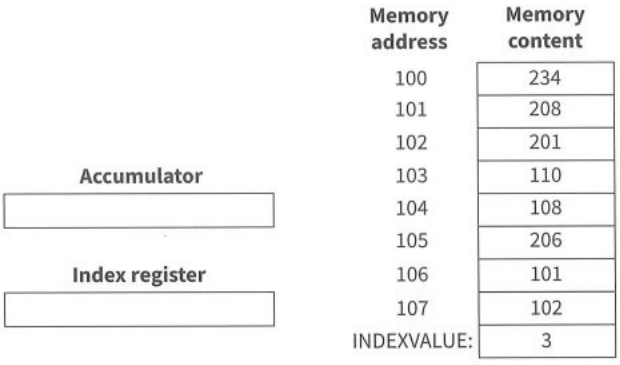


Figure 6.02 Example of some data stored in memory

The following shows some examples of the effect of an instruction or a sequence of instructions based on the memory content shown in Figure 6.02.

LDD 103	the value 110 is loaded into the accumulator
LDI 106	the value 208 from address 101 is loaded into the accumulator
STO 106	the value 208 is stored in address 106
LDD INDEXVALUE	the value 3 is loaded into the accumulator
MOV IX	the value 3 from the accumulator is loaded into the index register
LDX 102	the value 206 from address 105 is loaded into the accumulator

Input and output

There are two instructions provided for input or output. In each case the instruction has only an opcode; there is no operand.

- The instruction with opcode `IN` is used to store in the ACC the ASCII value of a character typed at the keyboard.
- The instruction with opcode `OUT` is used to display on the screen the character for which the ASCII code is stored in the ACC.

Comparisons and jumps

A program might need an unconditional jump or might need a jump if a condition is met. In the second case, a compare instruction is executed first. Table 6.08 shows the format for these types of instruction.

Instruction opcode	Instruction operand	Explanation
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with the number n
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False

Table 6.08 Jump and compare instruction formats

Note that the comparison is restricted to asking if two values are equal.

The result of the comparison is recorded by a flag in the status register. The execution of the conditional jump instruction begins by checking whether or not the flag bit has been set. This jump instruction does not cause an immediate jump. This is because a new value has to be supplied to the program counter so that the next instruction is fetched from this newly specified address. The incrementing of the program counter that took place automatically when the instruction was fetched is overwritten.

Arithmetic operations

There are no instructions for general-purpose multiplication or division. General-purpose addition and subtraction are catered for. Table 6.09 contains the instruction formats used for arithmetic operations.

Instruction opcode	Instruction operand	Explanation
ADD	<address>	Add the contents of the given address to the ACC
ADD	#n	Add the denary number n to the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
SUB	#n	Subtract the denary number n from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)

Table 6.09 Instruction formats for arithmetic operations

Figure 6.03 shows a program to find out how many times 5 divides into 75.

The following should be noted concerning the program.

- The first three instructions initialise the count and the sum.
- The instruction in address 103 is the one that is returned to in each iteration of the loop; in the first iteration it is loading the value 0 into the accumulator when this value is already stored but this cannot be avoided.

Memory address	Memory content
100	LDD 200
101	STO 202
102	STO 203
103	LDD 202
104	INC ACC
105	STO 202
106	LDD 203
107	ADD 201
108	STO 203
109	CMP 204
110	JPN 103
111	LDD 202
112	OUT
113	END
200	0
201	5
202	
203	
204	75

Figure 6.03 A program to calculate the result of dividing 75 by 5

- The next three instructions are increasing the count by 1 and storing the new value.
- Instructions 106 to 108 add 5 to the sum.
- Instructions 109 and 110 check to see if the sum has reached 75 and if it has not the program begins the next iteration of the loop.
- Instructions 111 to 113 are only used when the sum has reached 75 which causes the value 15 stored for the count to be output.

Shift operations

There are two shift instructions available:

- `LSL #n`
  where the bits in the accumulator are shifted logically n places to the left
- `LSR #n`
  where the bits are shifted to the right.

In a **logical shift** no consideration is given as to what the binary code in the accumulator represents. Because a shift operation moves a bit from the accumulator into the carry bit



KEY TERM

Logical shift: where bits in the accumulator are shifted to the right or to the left and a zero moves into the bit position vacated

in the status register this can be used to examine individual bits. For a left logical shift, the most significant bit is moved to the carry bit, the remaining bits are shifted left and a zero is entered for the least significant bit. For a right logical shift, it is the least significant bit that is moved to the carry bit and a zero is entered for the most significant bit.

If the accumulator content represents an unsigned integer, the left shift operation is a fast way to multiply by two. However, this only gives a correct result if the most significant bit is a zero. For an unsigned integer the right shift represents integer division by two. For example, consider:

00110001 (denary 49) gives if right shifted 00011000 (denary 24)

The remainder from the division can be found in the carry bit. Again, the division will not always give a correct result; continuing right shifts will eventually produce a zero for every bit. It should be apparent that a logical shift cannot be used for multiplication or division by two when a signed integer is stored. This is because the operation may produce a result where the sign of the number has changed.

As indicated earlier, only the two logical shifts are available for the simple processor considered here. However, in more complex processors there is likely to be a **cyclic shift** capability. Here a bit moves off one end into the carry bit then one step later moves in at the other end. All bit values in the original code are retained. Left and right **arithmetic shifts** are also likely to be available. These work in a similar way to logical shifts, but are provided for the multiplication or division of a signed integer by two. The sign bit is always retained following the shift.

KEY TERMS

Cyclic shift: similar to a logical shift but bits shifted from one end reappear at the other end

Arithmetic shift: uses the shift to carry out multiplication or division of a signed integer stored in the accumulator

Bitwise logic operation

The options for this are described in Table 6.10.

Instruction opcode	Instruction operand	Explanation
AND	#Bn	Bitwise AND operation of the contents of ACC with the binary number n
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#Bn	Bitwise XOR operation of the contents of ACC with the binary number n
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#Bn	Bitwise OR operation of the contents of ACC with the binary number n
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>

Table 6.10 Bitwise logical operation instructions

The operand for a bitwise logic operation instruction is referred to as a mask because it can effectively cover some of the bits and only affect specific bits. Some examples of their use are given in Chapter 7 (Section 7.03).

6.07 Further consideration of assembly language instructions

Register transfer notation

Section 6.01 introduced an extension to register transfer notation. We can use this to describe the execution of an instruction. For example, the `LDD` instruction is described by:

```
ACC ← [[CIR(15:0)]]
```

The instruction is in the CIR and only the 16-bit address needs to be examined to identify the location of the data in memory. The contents of that location are transferred into the accumulator.

TASK 6.02

Use register transfer notation to describe the execution of an LDI instruction.

Computer arithmetic

In Chapter 1 (Section 1.03) we saw that computer arithmetic could lead to an incorrect answer if overflow occurred. In Chapter 5 (Section 5.02) we saw the possible uses of the Status Register. The following worked example illustrates how the values stored in the Status Register can identify a specific overflow condition.

The use of the following three flags is required:

- the carry flag, identified as C, which is set to 1 if there is a carry
- the negative flag, identified as N, which is set to 1 if a result is negative
- the overflow flag, identified as V, which is set to 1 if overflow is detected.

WORKED EXAMPLE 6.01

Using the status register during an arithmetic operation

1 Consider the addition of two positive values where the sum of the two produces an answer that is too large to be correctly identified with the limited number of bits used to represent the values. For example, Figure 6.04 shows what happens if we use an eight-bit binary integer representation and attempt to add denary 66 to denary 68.

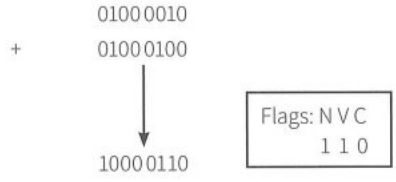


Figure 6.04 An attempted addition of denary 66 to denary 68

The answer produced is denary −122. Two positive numbers have been added to get a negative number. This impossibility is detected by the combination of the negative flag and the overflow flag being set to 1. The processor examines the flags, identifies the problem and generates an interrupt.

(Continued)

2 Consider using the same eight-bit binary integer representation but this time we add two negative numbers (−66 and −68 in denary). The result is shown in Figure 6.05.

```
  1011 1100
+ 1011 1110
      |
      v
(1)0111 1010
```

Flags: N V C
0 1 1

Figure 6.05 An attempted addition of denary −66 to denary −68

We get the answer +122. This impossibility is detected by the combination of the negative flag not being set and both the overflow and the carry flag being set to 1.

Extension Question 6.01

Carry out a comparable calculation for the addition in binary of −66 to +68. What do you think the processor should do with the carry bit?

Tracing an assembly language program

One way of checking to see if an assembly language program has errors is to carry out a dry (practice) run. The main feature of this will be to check how the contents of the accumulator change as the program runs. The following two worked examples illustrate the process.

WORKED EXAMPLE 6.02

Tracing an assembly language program

For this example the trace table needs a column for the accumulator, two for memory locations and one for the output.

The tracing is based on an initial user input of 15, a second input of 27 and a final input of 31.

The program is shown in Figure 6.06.

100	IN
101	STO 200
102	IN
103	STO 201
104	IN
105	ADD 200
106	STO 200
107	ADD 201
108	INC ACC
109	OUT
110	END

Figure 6.06 The assembly language program

(Continued)

The completed trace table is shown in Figure 6.07

Accumulator	Memory location 200	Memory location 201	Output
15			
	15		
27			
		27	
31			
46			
	46		
73			
74			
			74

Figure 6.07 The trace table showing the execution of the program

Note that in this presentation the decision has been made to use a new row in the trace table for each instruction in the program. This helps with checking. However, an alternative correct method is to enter a value in a column in the first available position. For example in the Memory location 200 column the first two rows could contain the 15 and 46. The other point to note is that if an instruction does not change an entry in a column it is not necessary to enter the value stored again. The trace table only needs to show activity; it does not have to record a complete set of values at each stage in the program execution.

WORKED EXAMPLE 6.03

Tracing an assembly language program

Some instructions for part of a program are contained in memory locations 100 upwards. Some 4-bit binary data values are stored in locations 200 upwards. For illustrative purposes the instructions are shown in assembly language form. At the start of a part of the program, the memory contents are as shown in Figure 6.08.

Address	Contents
100	LDD 200
101	INC ACC
102	ADD 201
103	CMP 202
104	JPE 106
105	DEC ACC
106	INC ACC
107	STO 203

Address	Contents
200	0001
201	0011
202	0101
203	

Figure 6.08 The contents of memory addresses before execution of the program begins

The completed trace table for this example is shown in Figure 6.09. Because the program contains a jump instruction it is necessary to record the values for the program counter as well as for the accumulator.

(Continued)

Program counter PC	Accumulator ACC	Memory location 203
100	1000	
101	0001	
102	0010	
103	0101	
104		
106		
107	0110	
108		0110

Figure 6.09 The contents of the program counter and accumulator during program execution

The entries in the table can be explained as follows.

- The first row shows the stored value before execution of this part of the program. There will be a value in the accumulator resulting from an earlier instruction.
- The second row shows the result of the execution of the instruction in location 100 which loads a value into ACC; this is followed by the PC being automatically incremented.
- The next two rows show the value being changed in the ACC by the instructions in 101 and 102 and the automatic incrementing of the PC each time.
- The fifth row has no new value in ACC because only a comparison is being done but there is an automatic increment of the PC.
- The sixth row shows a new value in the PC which has resulted from the execution of the jump instruction which tested for equality and found it to be True.
- The seventh row shows the result of the instruction in location 106 which has incremented the ACC.
- The final row shows the value stored in location 203.

Question 6.01

Can you follow through the changes in the trace table for Worked Example 6.03? Could it be possible for the program to change the content in one of the memory locations 100–107 during execution?

TASK 6.03

Without looking at the explanations provided, trace the assembly language program show in Table 6.01. Use a value of 3 for MAX and then 7, 8 and 9 as input values.